

使用 Ollama 建立本地 LLM

從本地模型到 Open WebUI、OpenCode、Continue 與本地工作流

Local LLM

Ollama

OpenAI-compatible API

Coding Assistant

核心問題：什麼時候應該用本地 LLM？如何把它安全接到本地應用？

本節學習目標

1. 理解本地 LLM 和雲端 LLM 的差異
2. 安裝並測試 Ollama
3. 用 CLI 和 API 呼叫本地模型
4. 把 Ollama 接到本地應用：Open WebUI、OpenCode、Continue
5. 選擇合適模型與硬體配置
6. 建立安全邊界：資料、網路、記錄、模型能力

一、為什麼要使用本地 LLM ？

不是取代雲端模型，而是多一種部署選項

本地 LLM 適合什麼？

適合

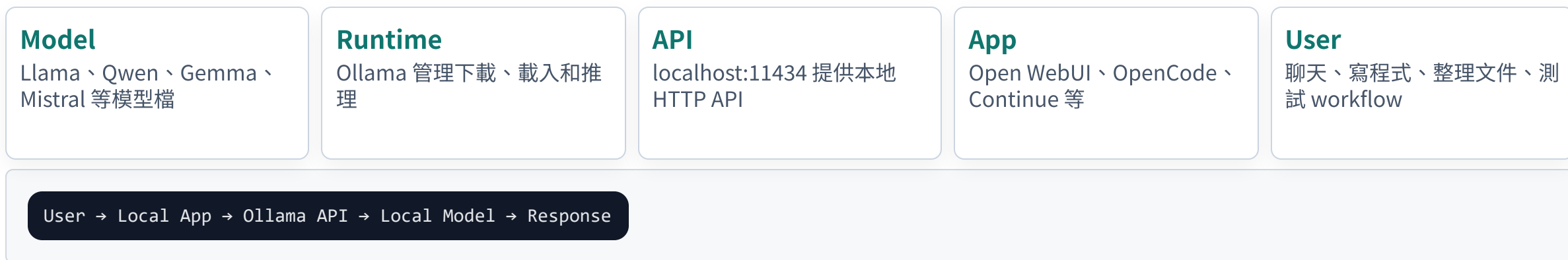
- 教學示範與離線練習
- 一般文字摘要與改寫
- 測試 prompt 與 agent workflow
- 敏感資料進入雲端前的去識別化處理
- coding assistant 的本地原型測試

不適合直接承擔

- 高風險法律或執法判斷
- 需要最新網上資料的任務
- 大型長文檔精確推理
- 需要穩定 tool calling 的生產系統
- 正式文件無人覆核輸出

本地執行不等於自動安全：prompt、模型檔、應用記錄、瀏覽器插件和開放連接埠仍可能留下資料痕跡。

本地 LLM 架構



Ollama 的官方 API 支援 generate、chat、pull model、list models、embeddings、structured outputs 等端點。

本地 vs 雲端：簡單比較

面向	本地 LLM	雲端 LLM
資料控制	主要留在本機或內網	需送到服務商
成本	硬體成本為主	API 或訂閱成本
速度	取決於 CPU/GPU/模型大小	取決於服務與網路
能力	小模型較常見，能力有限	通常較強、更新快
管理	需自行安裝、更新、配置	由服務商維護
適合	教學、原型、低風險本地任務	高能力推理、最新模型、穩定服務

二、安裝與測試 Ollama

先確認本地模型能跑，再接應用

安裝步驟

1. 到 Ollama 官方網站下載：<https://ollama.com/>
2. 安裝 Windows / macOS / Linux 版本
3. 開啟終端機確認服務可用

```
ollama --version
```

常用檢查：

```
ollama list  
ollama ps
```

本地 API 預設位址：

```
http://localhost:11434
```

Windows 安裝後通常會在背景執行；Linux / server 環境可用 service 或手動啟動。

拉取與執行模型

模型名稱通常是：

```
model:tag
```

常見示例：

```
ollama pull llama3.2:3b  
ollama run llama3.2:3b
```

其他可試模型：

```
ollama pull qwen2.5:7b  
ollama pull gemma3:4b  
ollama pull mistral:7b
```

模型名稱與可用 tag 會變動；上課前應到 `<https://ollama.com/library>` 確認。硬體不足時，優先選 1B、3B、4B 或 7B 級別模型。

第一個本地 Prompt

```
ollama run llama3.2:3b
```

測試 prompt：

```
請用繁體中文，用 5 點列出本地 LLM 和雲端 LLM 的差異。
```

離開互動模式：

```
/bye
```

觀察重點：

1. 回答速度
2. 中文能力
3. 是否遵守格式
4. 是否會編造不存在的功能
5. 是否需要更換模型

三、Ollama API 與本地服務模式

讓本地模型從命令列走向可被應用呼叫的服務

用 API 呼叫 Ollama

Ollama 可透過本地 HTTP API 呼叫模型：

```
curl.exe http://localhost:11434/api/generate `
-H "Content-Type: application/json" `
-d "{\"model\":\"llama3.2:3b\",\"prompt\":\"用繁體中文解釋 local LLM。\",\"stream\":false}"
```

Chat endpoint：

```
POST http://localhost:11434/api/chat
```

OpenAI-compatible endpoint 常用於應用整合：

```
http://localhost:11434/v1
```

結構化輸出測試

```
curl.exe http://localhost:11434/api/generate `
-H "Content-Type: application/json" `
-d '{"model":"llama3.2:3b","prompt":"列出三個本地 LLM 風險，請用 JSON 回答。","format":"json","stream":false}'
```

教學重點：

1. 本地模型也可以被程式呼叫
2. 可以要求 JSON 或 schema 格式
3. 小模型未必穩定遵守格式
4. 生產用途仍要加 parser、validator、retry

四、Ollama 與不同本地應用整合

同一個本地模型，可以服務不同工具

可搭配的本地應用

應用	類型	用途
Open WebUI	本地 ChatGPT-like UI	聊天、模型管理、RAG、多人使用
OpenCode	終端 coding agent	讀寫程式碼、執行開發任務
Continue	VS Code / JetBrains coding assistant	補全、聊天、repo 問答
LM Studio	本地模型 GUI / server	下載模型、聊天、OpenAI-compatible server
AnythingLLM	本地知識庫 / RAG app	文件問答、知識庫助理
LibreChat	多 provider chat UI	自架聊天介面，接多種 provider
Page Assist	瀏覽器 extension	用 Ollama 做本地網頁助理
Dify / LangChain / LlamaIndex app	開發框架	建立本地 prototype 或 RAG workflow

實際支援情況會隨版本變動；上課前應檢查每個工具的 provider 文件。

Open WebUI：本地 Chat 介面

用途：

1. 給非技術學員一個類 ChatGPT 介面
2. 連接 Ollama 本地模型
3. 管理聊天、模型、知識庫與 RAG
4. 適合作為課堂 demo UI

常見架構：

```
Browser → Open WebUI → Ollama → Local Model
```

Docker 示例：

```
docker run -d -p 3000:8080 \
-v open-webui:/app/backend/data \
--name open-webui \
ghcr.io/open-webui/open-webui:main
```

如 Docker 內的 Open WebUI 要連本機 Ollama，需按作業系統設定 `OLLAMA\_BASE\_URL`。

Continue：IDE 內使用本地模型

Continue 可在 VS Code / JetBrains 中連接 Ollama，用於：

1. 解釋程式碼
2. 產生小段函式
3. 對 repo 提問
4. 建議重構方向
5. 搭配 embeddings 做本地 codebase context

設定概念：

```
provider: ollama  
model: llama3.2:3b  
apiBase: http://localhost:11434
```

Continue 適合示範「IDE 內 local assistant」；OpenCode 適合示範「終端 agent 工作流」。

LM Studio 與其他本地工具

工具	怎樣和 Ollama / local model 協作	注意
LM Studio	可作本地 GUI 與 OpenAI-compatible server	與 Ollama 是不同 runtime
AnythingLLM	可接本地模型做文件知識庫 / RAG	注意文件索引與儲存位置
LibreChat	可自架 chat UI 接多 provider	需要設定 provider 與權限
Page Assist	瀏覽器 extension，可接 Ollama	注意網頁內容與本地資料邊界
Dify / LangChain	可把 Ollama 當 local model endpoint	適合 prototype，不等於生產部署

應用整合共同模式

1. App

Open WebUI / Continue /
OpenCode

2. Endpoint

http://localhost:11434 或
/v1

3. Model

指定 llama / qwen / coder
model

4. Context

聊天、文件、repo、知識庫

5. Safety

權限、資料、log、人工覆核

接入應用前先確認三件事：應用會讀取什麼資料、會把資料存在哪裡、是否有權限修改檔案或連外。

OpenCode：用本地模型做 coding agent

OpenCode 可透過 Ollama 使用本地模型。核心設定是把 provider 指向 Ollama 的 OpenAI-compatible endpoint：

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Ollama (local)",
      "options": {
        "baseUrl": "http://localhost:11434/v1"
      },
      "models": {
        "llama3.2:3b": {
          "name": "Llama 3.2 3B Local"
        }
      }
    }
  }
}
```

Coding agent 會讀取或修改本地檔案。練習時應使用測試 repo，並先看 diff 再接受變更。

五、OpenCode Agent 與 Skills

Agent 定義誰來做事，Skills 定義怎樣完成任務

OpenCode 的 Agent 概念

OpenCode 不是單純聊天工具，而是 terminal-based coding agent。它會在專案資料夾中讀取檔案、搜尋內容、提出計劃，必要時修改檔案或執行命令。

Agent 類型	用途	典型風險
Plan	只規劃，不直接修改	計劃可能不完整
Build	可實作、編輯檔案、執行工具	可能改錯檔案或執行高風險命令
General	通用 subagent	需要清楚任務邊界
Explore	探索 codebase / 資料夾	可能讀到不應讀的檔案
Scout	搜尋與定位資訊	結果仍需人工判斷

官方文件中，primary agents 是主要對話代理；subagents 可由主要代理呼叫，也可用 `@` mention 手動呼叫。

OpenCode 常用操作

啟動：

```
cd path\to\project  
opencode
```

初始化專案：

```
/init
```

`/init` 會讓 OpenCode 分析專案，並建立 `AGENTS.md`，記錄專案結構、慣例和 agent 應遵守的指引。

常用互動：

請閱讀 `README`，說明這個專案用途。不要修改任何檔案。

請先建立修改計劃，不要直接編輯檔案。

`@explore` 找出這個專案的入口檔案和主要設定檔。

Plan Mode vs Build Mode

模式	何時使用	建議 prompt
Plan	需求未清楚、怕 agent 亂改、需要先審查方案	請先提出計劃，不要修改檔案
Build	已同意方案，要實作小修改	按上述計劃修改，完成後列出 diff 摘要

建議流程：

1. Ask

先問專案結構或問題原因

2. Plan

要求提出實作計劃

3. Review

人工檢查計劃

4. Build

批准小範圍修改

5. Verify

看 diff、跑測試、必要時
undo

教學時不要一開始就讓 agent 自由修改整個專案。先從只讀任務、計劃任務、小範圍修改開始。

OpenCode Agent 權限控制

OpenCode 使用 `permission` 控制工具是否可自動執行、需詢問或被拒絕：

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "*": "ask",
    "read": "allow",
    "grep": "allow",
    "glob": "allow",
    "edit": "ask",
    "bash": {
      "*": "ask",
      "git status *": "allow",
      "git diff *": "allow",
      "git commit *": "deny",
      "git push *": "deny",
      "rm *": "deny"
    }
  }
}
```

權限值：

- `allow`：直接執行
- `ask`：先問使用者
- `deny`：禁止

自訂 Agent：只讀 Reviewer

可以在 config 中建立只讀 subagent，用於 code review 或文件檢查：

```
{
  "$schema": "https://opencode.ai/config.json",
  "agent": {
    "readonly-reviewer": {
      "mode": "subagent",
      "description": "Review code and documents without editing files.",
      "permission": {
        "read": "allow",
        "grep": "allow",
        "glob": "allow",
        "edit": "deny",
        "bash": "ask",
        "webfetch": "deny"
      }
    }
  }
}
```

使用：

```
@readonly-reviewer 請檢查這份 README 是否清楚，不要修改檔案。
```

Agent 也可用 Markdown 檔配置，放在 OpenCode 的 agents 設定目錄或專案 `.opencode/agents/` 中。

OpenCode Skills 是什麼？

Skills 是一組可重用的工作指引，讓 agent 在需要時載入特定任務流程。

概念	說明
Agent	定義「誰」來做事，例如 planner、reviewer、explorer
Skill	定義「怎樣做某類任務」，例如文件審查、PDF 處理、slides 檢查
MCP	提供「可呼叫的外部工具」，例如 filesystem、GitHub、資料庫、法例搜尋

簡單說：Agent 是角色，Skill 是操作手冊，MCP 是工具箱。

Skills 為什麼不是普通 Prompt ？

普通 prompt 通常只是一段文字；Skill 是可被 agent 發現、選擇和分層載入的 workflow package 。

層級	何時載入	內容
Level 1	掃描 skills 時	<code>name</code> 、 <code>description</code> 等 metadata
Level 2	判斷相關時	<code>SKILL.md</code> 主體：流程、規則、例子
Level 3	真正需要時	<code>scripts/</code> 、 <code>references/</code> 、 <code>assets/</code>

好處：

1. 不用把所有專門知識一次塞入 context
2. 只在任務相關時載入細節
3. 可以把固定流程、參考資料和 script 放在同一個資料夾
4. 方便測試、維護和重用

Skill 資料夾結構

一個 skill 通常是一個資料夾，入口是 `SKILL.md`。

```
skills/  
└─ police-report-review/  
   └─ SKILL.md      # 必要: metadata + instructions  
   └─ scripts/      # 可選: parser、validator、自動化腳本  
   └─ references/    # 可選: SOP、格式規範、範例文件  
   └─ assets/        # 可選: 模板、表格、樣式、圖片
```

設計重點：

1. `SKILL.md` 保持短而清楚
2. 大量背景資料放 `references/`
3. 重複、可驗證的步驟放 `scripts/`
4. 所有外部工具使用仍要受 permission 控制

Skill Metadata：讓 Agent 選得中

`description` 是最重要的觸發訊號，要寫出使用場景和邊界。

```
---
name: police-report-review
description: Reviews police report drafts for factual clarity, missing verification, source labeling, and overclaiming risks. Use when checking incident summaries, complaint records, witness notes, or police document drafts. Do not use for legal conclusions.
license: MIT
compatibility: opencode; read-only review; no legal advice
---
```

壞例子：

```
description: Helps with police documents.
```

問題：太籠統、沒有 trigger、沒有任務邊界。

寫 Skill 前先定義 Use Case

不要先寫 `SKILL.md`，先定義 2-3 個具體場景。

欄位	例子
Use Case	報案摘要文件審查
Trigger	「檢查這份案件摘要」、「整理報案紀錄」、「看看是否有推論過度」
Steps	讀文件、標示已知事實、找未核實資料、檢查是否越界下結論
Tools	read / grep；必要時使用本地文件 MCP
Result	一份不改寫原文的審查清單

判斷是否值得做成 skill：

1. 是否重複出現？
2. 是否多步驟？
3. 是否容易忘記安全邊界？
4. 是否有固定輸出格式？

SKILL.md 主體建議結構

Police Report Review

When to use

Use when checking incident summaries, complaint records, witness notes, or draft reports.

Steps

1. Identify document type.
2. Extract known facts, unverified claims, source labels, and missing questions.
3. Flag legal conclusions, unsupported assumptions, and identity exposure.
4. Return a review table. Do not rewrite the whole report unless asked.

Output format

Issue	Location	Reason	Suggested question
-------	----------	--------	--------------------

Boundaries

Do not make legal conclusions. Do not invent missing facts. Mark uncertainty explicitly.

核心原則：流程要具體，輸出要固定，限制要明確。

Skill 可以怎樣用 Scripts ?

如果任務有重複、可驗證、容易出錯的步驟，可以放 script。

例子：PDF 文字抽取 skill

```
pdf-processing/  
├─ SKILL.md  
└─ scripts/  
    └─ parse_pdf.py
```

SKILL.md 可要求 agent：

1. 先確認 PDF 路徑是否存在
2. 使用 scripts/parse_pdf.py 抽取文字
3. 回傳 page-level JSON
4. 如有空白頁、掃描影像或解析錯誤，要明確標示

Skill 引用 script 不等於可以自動執行。是否可執行 script，仍由 OpenCode permission 和使用者批准控制。

Skills vs MCP：不要混淆

面向	Skill	MCP
本質	Markdown 指引與任務流程	外部工具協議
作用	告訴 agent 怎樣完成某類任務	讓 agent 呼叫工具或資料來源
形式	<code>SKILL.md</code> + 可選資料夾	MCP server process / remote endpoint
是否執行程式	不直接執行	server 可提供 tools
例子	文件審查流程、slides audit、PDF processing 指引	filesystem、GitHub、資料庫、法例搜尋
主要風險	指引錯誤、觸發錯誤、流程越界	讀寫檔案、連外、token、資料外洩

Skill 可以指示 agent 使用 MCP，但真正讀寫資料的是 MCP tool 或 OpenCode 內建工具。

Skill + MCP 的組合例子

任務：檢查教學資料夾中的 Markdown，產生安全與清晰度審查。

元件	負責
Agent	<code>readonly-reviewer</code> ，只讀審查
Skill	<code>doc-review</code> ，定義審查準則
MCP	<code>demo-files</code> ，只允許讀取 <code>./mcp-demo</code>

Prompt：

```
@readonly-reviewer 請使用 doc-review skill。  
如需要讀取檔案，只能使用 demo-files MCP。  
先列出 tool plan，等我確認後才讀取 notes.md。  
不要修改任何檔案。
```

這個組合把「誰做、怎樣做、用什麼工具」分開管理。

測試 Skill 是否真的有效

測試不只看 skill 能否載入，也要看它是否改善結果。

測試	目標	例子
Triggering	該觸發時觸發，不該觸發時不觸發	「檢查案件摘要」應觸發；「寫 Python」不應觸發
Functional	output 是否正確、穩定、能處理錯誤	是否標示未核實資料、是否避免法律結論
Performance	是否比不用 skill 更少重試、更少錯誤	比較 messages、tool calls、token、人工修正次數

常見修法：

1. Under-triggering：在 description 加 trigger phrase
2. Over-triggering：縮小 scope，加 negative boundary
3. 格式不穩：加入 output schema 和例子
4. 指令被忽略：把 critical rules 放前面，改成 numbered steps

OpenCode Skills 放在哪裡？

每個 skill 一個資料夾，裡面放 `SKILL.md`。

專案層級：

```
.opencode/skills/<name>/SKILL.md  
.claude/skills/<name>/SKILL.md  
.agents/skills/<name>/SKILL.md
```

全域層級：

```
~/.config/opencode/skills/<name>/SKILL.md  
~/.claude/skills/<name>/SKILL.md  
~/.agents/skills/<name>/SKILL.md
```

建議：

1. 課堂示範用 `.opencode/skills/`
2. 團隊共用 skill 放專案 repo
3. 個人常用 skill 放全域 config
4. 預設用 `permission.skill` 設為 `ask`

OpenCode Skills 課堂練習

任務：建立一個 `police-report-review` skill。

最低要求：

```
.opencode/skills/police-report-review/SKILL.md
```

`SKILL.md` 必須包括：

1. `name` 和具體 `description`
2. when to use / when not to use
3. review steps
4. output table format
5. safety boundaries：不下法律結論、不編造資料、標示未核實資訊

測試 prompt：

```
請使用 police-report-review skill 檢查這份去識別化案件摘要，列出已知事實、未核實資料、待補問題和不可下結論之處。
```

六、MCP 入門：讓 Agent 使用外部工具

Model Context Protocol 把模型和工具之間的連接標準化

Hugging Face Unit 1 的完整脈絡

這個 topic 整合 Hugging Face MCP Course Unit 1，而不只是 introduction 首頁：

頁面	學習重點
Introduction	MCP 為什麼重要：讓模型連接外部資料、工具與環境
Key Concepts	USB-C analogy、M×N integration problem、Host / Client / Server
Architecture	Host、Client、Server 的責任分工與 workflow
Communication Protocol	JSON-RPC 2.0、Requests / Responses / Notifications、stdio / HTTP+SSE
Capabilities	Tools、Resources、Prompts、Sampling 與 discovery
SDK / Clients	Python / TypeScript SDK、MCP Inspector、client config
HF MCP / Gradio	連接 Hugging Face Hub、Gradio app 變成 MCP server

MCP 的核心定位

MCP 是一個標準化協議，用來把 AI application 連接到外部工具、資料來源與執行環境。

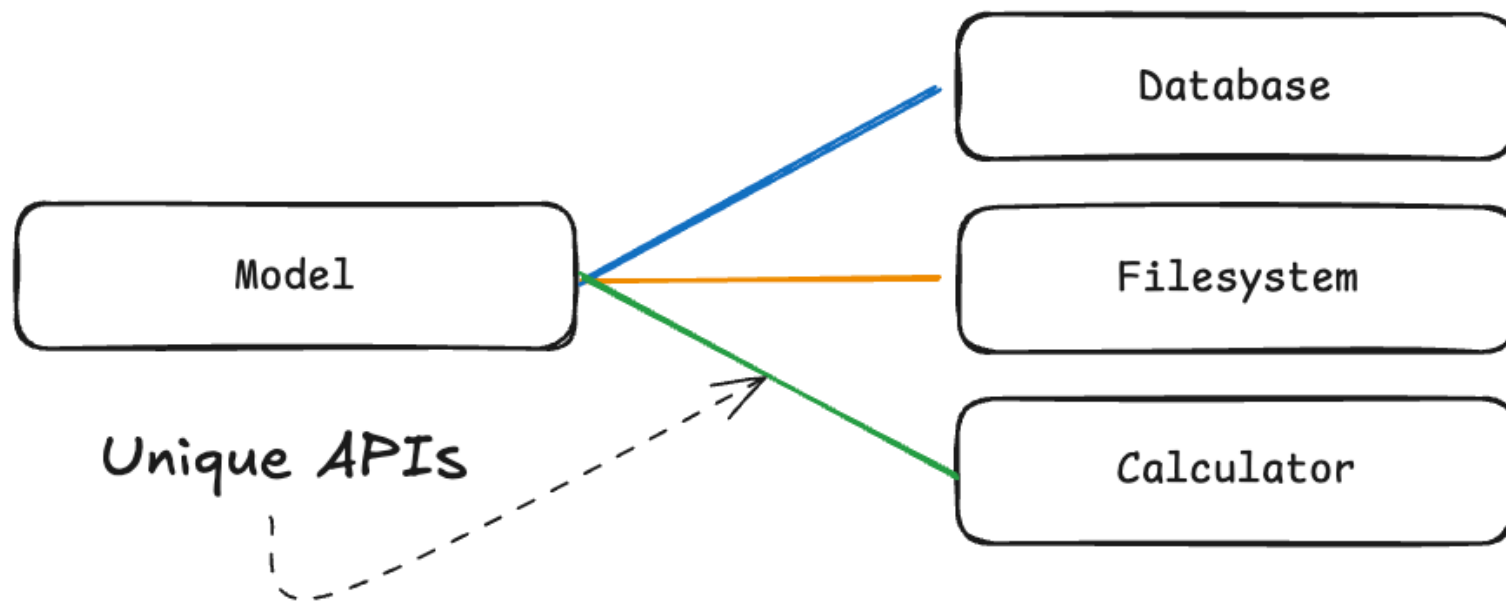
Hugging Face 課程把 MCP 類比成「AI applications 的 USB-C」：不是每個 app 和每個工具都各自寫一套接線，而是大家使用同一個連接標準。

MCP 想解決的不是「模型是否聰明」，而是：

1. 模型如何取得即時資料
2. 模型如何呼叫外部工具
3. 不同 AI 應用如何用同一套方式接工具
4. 工具供應者如何只實作一次，就可被多個 AI app 使用
5. 使用者如何保留權限與安全控制

M × N Integration Problem

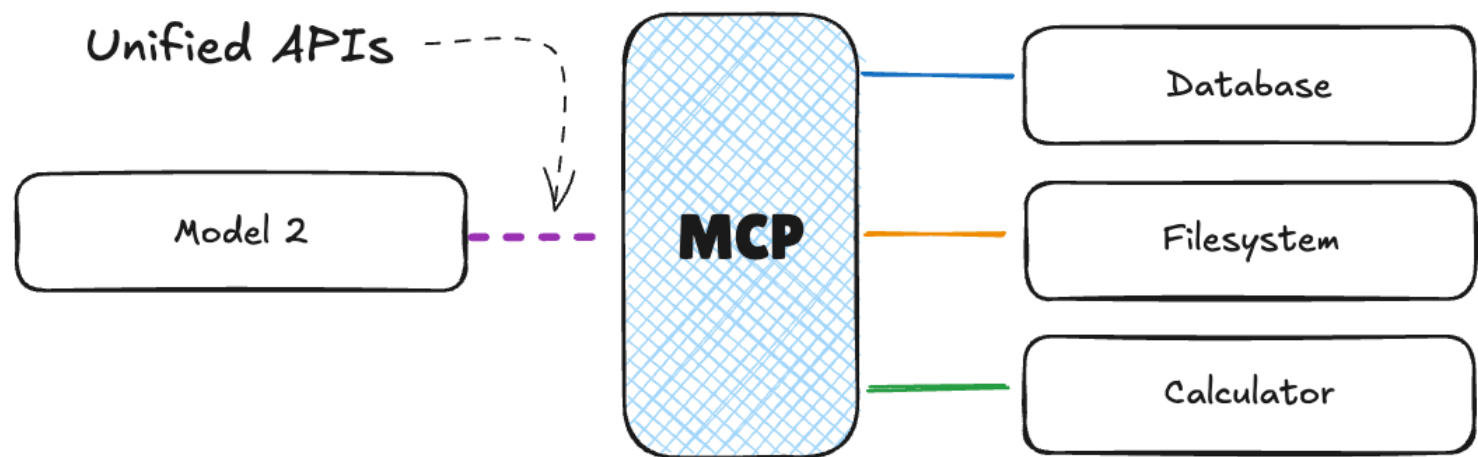
沒有 MCP 時，每個 AI application 都要分別接每個工具或資料庫。



Source: Hugging Face MCP Course, Unit 1 Key Concepts

MCP 把 $M \times N$ 變成 $M+N$

有 MCP 後，每個 AI application 實作一次 MCP client，每個工具或資料來源實作一次 MCP server。



重點：MCP 不是另一個模型，而是一個 interoperability protocol。它降低的是整合成本和維護成本。

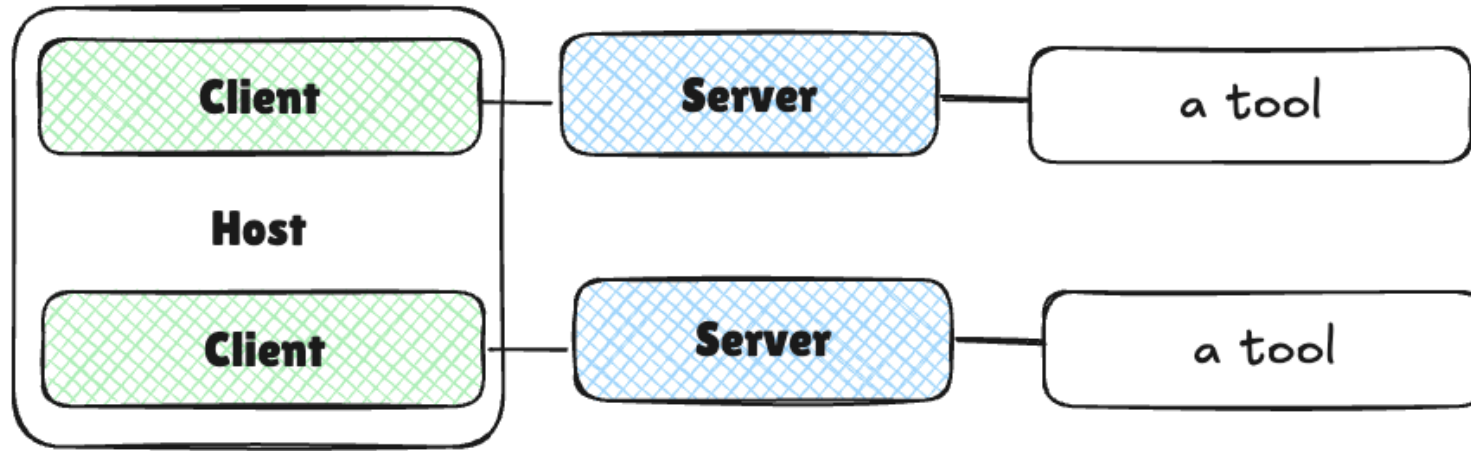
MCP 三個角色：Host / Client / Server



角色	說明	課堂例子
Host	使用者直接操作的 AI application	OpenCode、Cursor、Claude Desktop、Continue
Client	Host 內負責連接某一個 MCP server 的組件	OpenCode 內的 MCP client
Server	對外暴露 tools / resources / prompts 的程式或服務	filesystem server、Playwright server、HF MCP server

注意：很多文章會混用 client / host；嚴格來說 client 是 host 內的通訊組件。

MCP Architecture：一個 Host 可接多個 Server

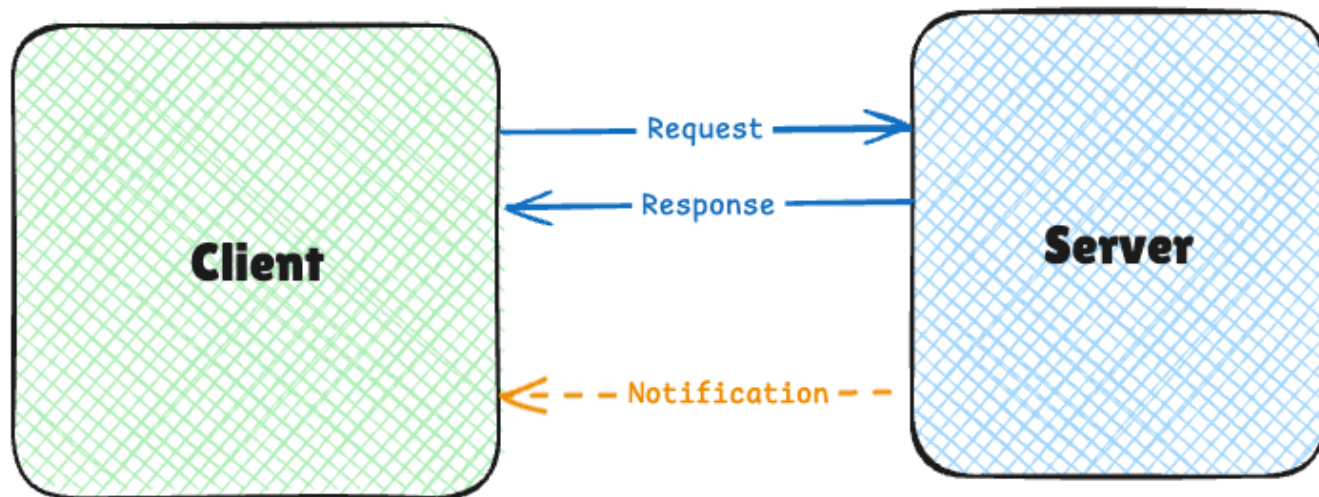


常見流程：

1. User 在 Host 提出任務
2. Host 用 LLM 判斷是否需要外部能力
3. Client 連接對應 MCP Server
4. Server 回傳可用 capability 清單
5. Host / LLM 選擇要呼叫的 capability
6. Server 執行後回傳結果
7. Host 把結果整合回回答或後續行動

MCP Communication Protocol

Hugging Face Unit 1 說明 MCP 底層使用 **JSON-RPC 2.0** 作為訊息格式。



訊息類型	用途
Request	Client 要求 Server 執行操作，例如 <code>tools/call</code>
Response	Server 回覆成功結果或錯誤
Notification	單向通知，不要求回覆，例如 progress update

Transport : Local 與 Remote MCP

Transport	適用	說明
<code>stdio</code>	本地工具	Host 啟動 server process，透過 stdin / stdout 溝通
<code>HTTP + SSE</code>	遠端服務	經 HTTP 連接，SSE 用於 server 推送更新
Streamable HTTP	新版遠端模式	需要串流時可升級，對 serverless 更友善

```
local stdio:  
OpenCode -> starts MCP server process -> stdin/stdout JSON-RPC  
  
remote HTTP:  
OpenCode -> HTTPS -> remote MCP server -> tool result
```

MCP Interaction Lifecycle

1. Initialization

client -> initialize -> server

server -> capabilities / version -> client

2. Discovery

client -> tools/list, resources/list, prompts/list

server -> available capabilities

3. Execution

client -> tools/call or resource read

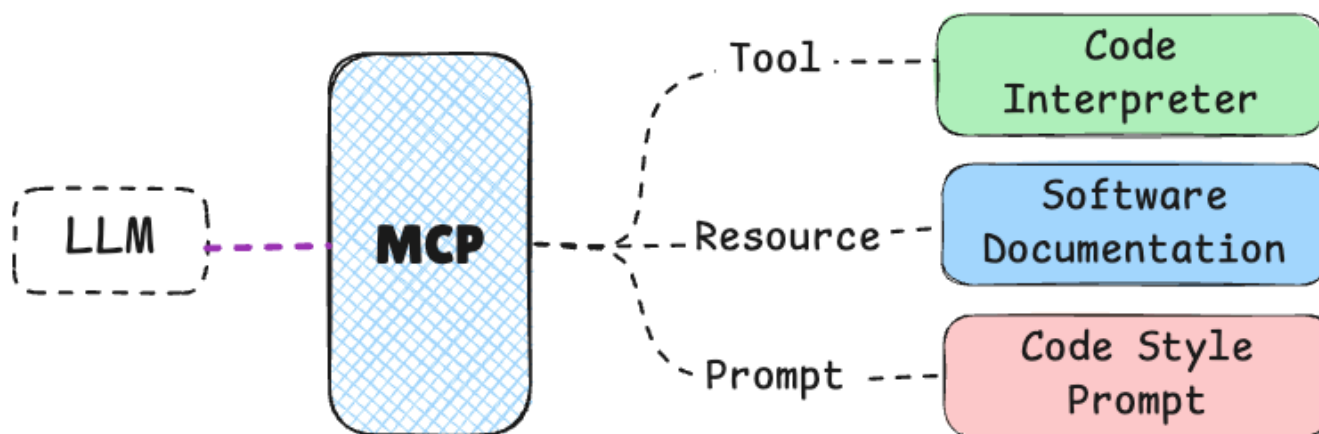
server -> result / error / progress notification

4. Termination

client -> shutdown / exit

所以 agent 不是「憑空知道工具存在」：它要先透過 discovery 知道 server 暴露了什麼 capability。

MCP Capabilities : 四種能力



Capability	誰控制	典型用途	風險
Tools	Model-controlled	查 API、送訊息、建立 ticket、執行計算	可能有 side effects
Resources	Application-controlled	讀文件、資料庫、設定、參考資料	通常 read-only
Prompts	User-controlled	預設工作流程、任務模板	低，但要避免錯誤模板
Sampling	Server-initiated	Server 要求 Host 再做一次 LLM 生成	需要人類確認

Capabilities 的 Discovery

MCP client 連上 server 後，可以動態查詢 server 有哪些能力：

```
tools/list  
resources/list  
prompts/list
```

這代表：

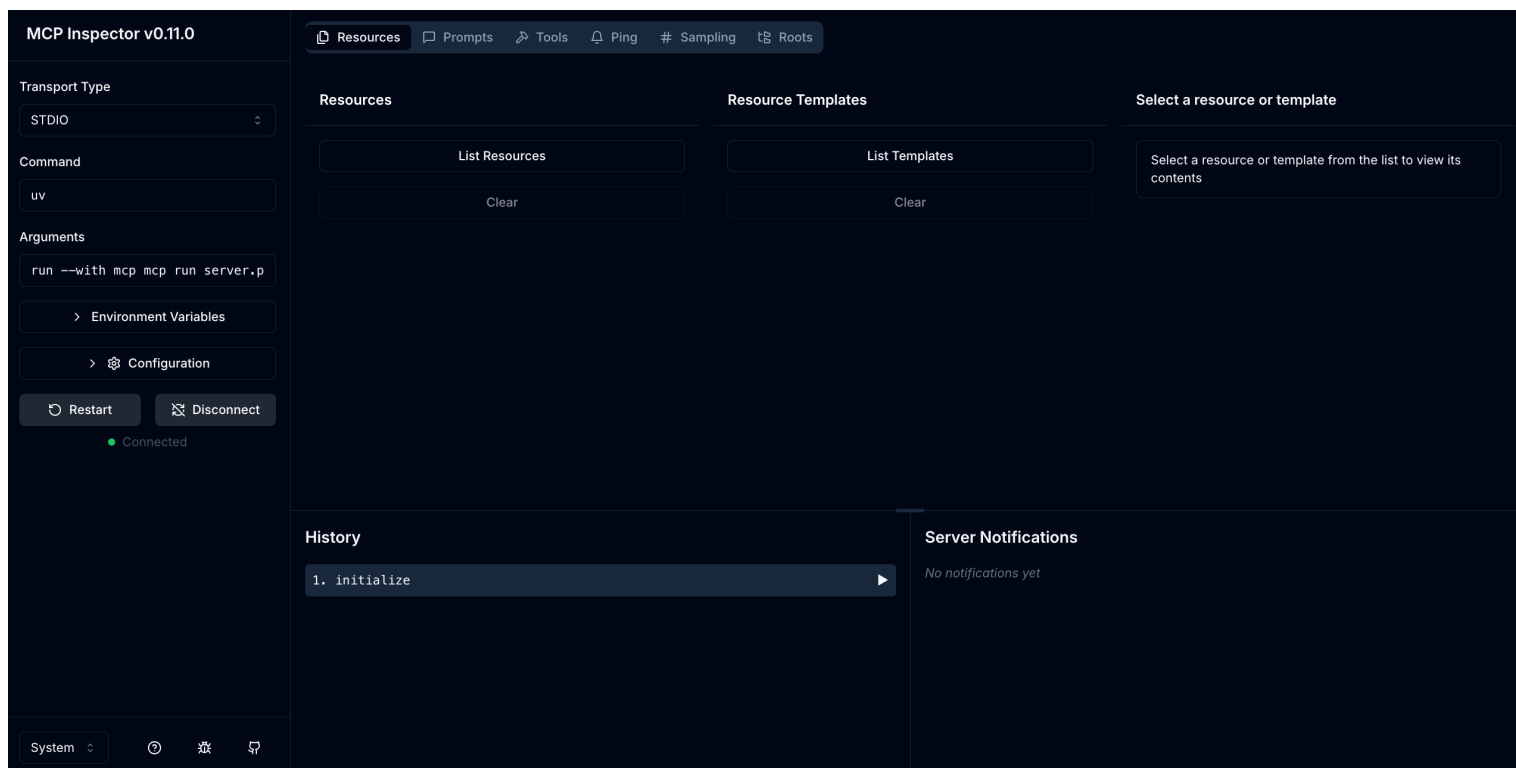
1. Host 不需要預先寫死每個工具的 schema
2. Server 可以新增工具，只要 client 重新 discovery
3. Agent 可以根據工具名稱、描述、參數 schema 決定是否呼叫
4. 使用者可以在呼叫前要求 agent 先說明 tool plan

Discovery 不是授權。看到工具存在，不代表應該自動執行；高風險 tool call 仍應要求確認。

MCP SDK 與 Inspector

MCP SDK 幫開發者處理底層協議：

1. protocol communication
2. capability registration and discovery
3. JSON serialization / deserialization
4. connection management
5. error handling



Python MCP Server 概念例子

Hugging Face Unit 1 用 weather service 示範 Tools、Resources、Prompts：

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Weather Service")

@mcp.tool()
def get_weather(location: str) -> str:
    return f"Weather in {location}: Sunny, 72F"

@mcp.resource("weather://{location}")
def weather_resource(location: str) -> str:
    return f"Weather data for {location}: Sunny, 72F"

@mcp.prompt()
def weather_report(location: str) -> str:
    return f"You are a weather reporter. Weather report for {location}?"

mcp.run()
```

教學重點：同一個 server 可以同時提供 tool、resource 和 prompt。

MCP Client 設定概念

Unit 1 說明 MCP host 通常用設定檔管理 server 連線。

```
{
  "servers": [
    {
      "name": "File Explorer",
      "transport": {
        "type": "stdio",
        "command": "python",
        "args": ["/path/to/file_explorer_server.py"]
      }
    }
  ]
}
```

遠端 server 則會使用 URL：

```
{
  "name": "Remote API Server",
  "transport": {
    "type": "sse",
    "url": "https://example.com/mcp-server"
  }
}
```

Hugging Face MCP Server

Hugging Face 提供 hosted MCP server，可讓 MCP-compatible assistant 搜尋 Hub 資源。

**Elliott Coyac**
coyotte508

Switch ▾

Profile

Account

Authentication

Organizations

Billing

Access Tokens

SSH and GPG Keys

Inference Providers

Webhooks

Papers

Notifications

Jobs NEW

Local Apps and Hardware

Gated Repositories

Content Preferences

Connected Apps

MCP

Theme

Upgrade to  Pro

MCP Server

Connect your MCP client and configure which tools are available in your Hugging Face Hub MCP server.

 **Get PRO** to get 25 minutes of daily compute usage on ZeroGPU Spaces (H200 hardware), including via MCP.

Setup with your AI assistant

 HuggingChat

 VSCode

 Cursor

 ChatGPT

 Codex

 Gemini CLI

 Claude

 Claude Code

 LM Studio

Other Client

Enable in your HuggingChat MCP servers settings, available by default with no configuration needed.

 **Open HuggingChat** to start chatting with 110+ available open weights models.

[Share feedback](#)

Spaces Tools

Any Spaces added here will be instantly available in your MCP client.

☒ **Dynamic Spaces** Experimental
Dynamically call MCP Spaces at runtime

☐ **Remove Embedded Images**
Remove embedded images generated by Gradio Spaces

☐ **MCP-UI Support** Experimental
Embed Gradio Spaces directly in your mcp-ui client

[Browse MCP Spaces](#)

Built-in Tools

Connect your AI assistant to the Hugging Face ecosystem.

☒ **Spaces Semantic Search**
Find the best AI Apps via natural language queries

☒ **Model Search**
Search for ML models with filters for task, library, etc...

☒ **Documentation Semantic Search**
Search the Hugging Face documentation library

☒ **Hub Repository Details**
Get detailed information about Models, Datasets and Spaces

☐ **Include repository README files**
Allow repository README files to be included in the results

☒ **Papers Semantic Search**
Find ML Research Papers via natural language queries

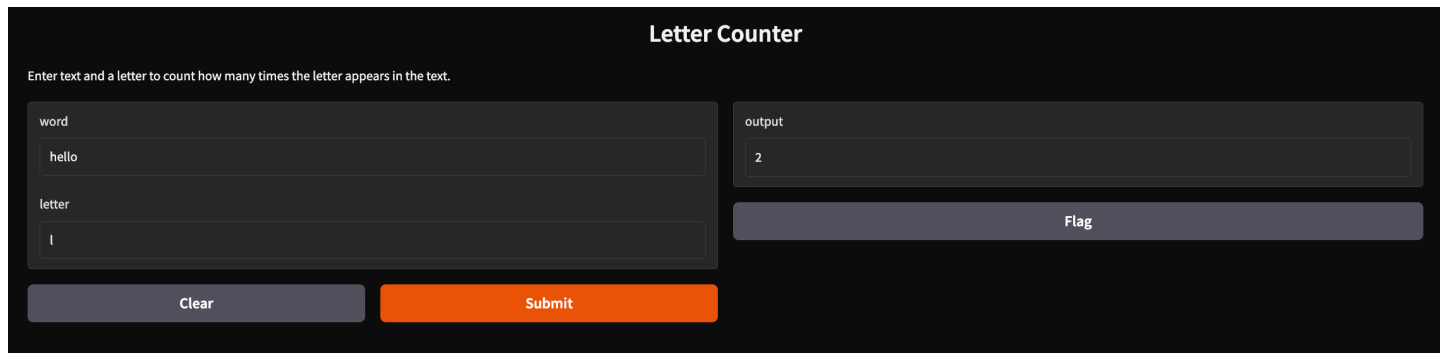
☒ **Dataset Search**
Search for datasets with filters for author, tags, etc...

☐ **Run and Manage Jobs**
Run, monitor and schedule jobs on Hugging Face infrastructure

[View MCP documentation](#)

Gradio MCP：把 demo 變成 tool

Gradio 可以用很少 Python code 建立 web UI，也可以把 function 暴露成 MCP tool。



The screenshot shows a web application titled "Letter Counter". At the top, it says "Enter text and a letter to count how many times the letter appears in the text." Below this, there are two input fields: "word" containing "hello" and "letter" containing "l". To the right of these is an "output" field containing the number "2". At the bottom, there are two buttons: a grey "Clear" button and an orange "Submit" button.

```
import gradio as gr

def letter_counter(word: str, letter: str) -> int:
    return word.lower().count(letter.lower())

demo = gr.Interface(
    fn=letter_counter,
    inputs=["textbox", "textbox"],
    outputs="number",
    title="Letter Counter"
)

demo.launch(mcp_server=True)
```

Gradio MCP 背後做了什麼？

當 Gradio 使用 `mcp_server=True`：

1. Gradio function 自動轉成 MCP tools
2. input components 轉成 tool argument schema
3. output components 決定 response format
4. Gradio server 同時處理 web UI 與 MCP protocol messages
5. 透過 HTTP+SSE / JSON-RPC 和 MCP client 溝通

可用 endpoint：

```
http://your-server:port/gradio_api/mcp/sse  
http://your-server:port/gradio_api/mcp/schema
```

這很適合課堂：學生可以先做一個普通 Gradio demo，再把它變成 agent 可呼叫的 MCP tool。

MCP 解決什麼問題？

沒有 MCP 時，每個 AI app 都要為每個工具寫一套整合：

```
App A → GitHub  
App A → Filesystem  
App A → Database  
App B → GitHub  
App B → Filesystem  
App B → Database
```

這會變成 $M \times N$ 個整合問題。

MCP 的目標是把它變成標準介面：

```
AI App / Agent → MCP Client → MCP Server → Tool or Data Source
```

Hugging Face MCP course 把 MCP 介紹成 AI 應用與外部工具之間的標準化連接方式，類似讓不同工具有共同插頭。

MCP 的三個角色

角色	說明	例子
Host	使用 MCP 的 AI 應用	OpenCode、Claude Desktop、IDE agent
Client	Host 裡負責連接 MCP server 的部分	OpenCode 的 MCP client
Server	提供工具、資料或 prompt 的外部服務	filesystem server、GitHub server、database server

資料流：

```
User request
→ Host decides it needs a tool
→ Client calls MCP server
→ Server runs tool or retrieves context
→ Observation returns to agent
```

MCP Server 可以提供什麼？

能力	用途	例子
Tools	執行動作或查詢	讀檔、搜尋、查資料庫、呼叫 API
Resources	提供可讀取資料	文件、設定、資料表、知識庫片段
Prompts	提供可重用 prompt template	code review prompt、文件整理 prompt

教學重點：

1. Tool 會執行動作，風險最高
2. Resource 偏讀取資料，也要注意敏感內容
3. Prompt 是模板或流程，不等於工具權限

OpenCode 使用 MCP：概念

MCP (Model Context Protocol) 讓 OpenCode 接入外部工具。OpenCode 支援：

MCP 類型	說明	例子
Local MCP	OpenCode 啟動本機 MCP server process	檔案工具、資料庫工具、測試工具
Remote MCP	連到遠端 MCP server URL	Jira、GitHub、內部 API
OAuth MCP	遠端 server 要求登入授權	需要 token 管理

加入 MCP 後，MCP tools 會和 OpenCode 內建工具一起提供給 LLM 使用。

MCP 等於給 agent 更多工具。每加一個 MCP，都要問：它能讀什麼？能改什麼？會不會連到外部服務？

MCP 要去哪裡下載？

MCP 不是單一軟體，而是一個協議。你通常下載或啟動的是「MCP server」。

常見來源：

來源	適合
官方 MCP servers GitHub	filesystem、git、fetch、memory 等基礎工具
npm packages	Node.js MCP server，常用 <code>npx</code> 啟動
PyPI / uvx	Python MCP server
Docker images	隔離執行 server
工具官方文件	GitHub、Jira、Postgres、browser automation 等整合
公司內部 repo	內部資料庫、文件庫、SOP 查詢工具

下載 MCP 的實際意思是：選一個 MCP server，安裝它的執行環境，然後在 OpenCode config 中註冊。

MCP Server 安裝方式

方式 1：用 `npx` 直接啟動 Node MCP server

```
npx -y @modelcontextprotocol/server-everything
```

方式 2：用 `uvx` 啟動 Python MCP server

```
uvx some-mcp-server
```

方式 3：用 Docker 啟動

```
docker run --rm -i some-mcp-server
```

方式 4：從 GitHub clone 後本地執行

```
git clone https://github.com/modelcontextprotocol/servers
```

實際 Local MCP 例子：Filesystem

用途：讓 OpenCode 透過 MCP 讀取或操作指定資料夾中的檔案。

先建立一個教學用資料夾：

```
New-Item -ItemType Directory -Force .\mcp-demo
```

測試 MCP server 是否能啟動：

```
npx -y @modelcontextprotocol/server-filesystem .\mcp-demo
```

安全重點：

1. 只把 `.\mcp-demo` 暴露給 MCP
2. 不要直接暴露整個使用者目錄或課程根目錄
3. 不要放真實個資、token 或內部文件

OpenCode 設定 Filesystem MCP

在 `opencode.jsonc` 加入：

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "demo-files": {
      "type": "local",
      "command": [
        "npx",
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "./mcp-demo"
      ],
      "enabled": true,
      "timeout": 5000
    }
  }
}
```

這個設定的意思：OpenCode 可透過 `demo-files` MCP server 存取 `./mcp-demo`。

Filesystem MCP 課堂測試

先在 `mcp-demo` 放一個測試檔：

```
mcp-demo/notes.md
```

內容：

課堂筆記

本地 LLM 適合教學、原型測試和低風險資料整理。
正式文件仍需人工覆核。

在 OpenCode 中測試：

請使用 `demo-files` MCP 讀取 `notes.md`，摘要成 3 點。
不要修改任何檔案。

進階測試：

請先說明你會呼叫哪個 MCP tool 和目的，等我確認後再讀取 `notes.md`。

使用 MCP 的完整步驟

1. 選工具

決定要接本地工具、資料庫、文件或遠端 API

2. 配 server

在 opencode.jsonc 的 mcp 區塊加入 server

3. 啟用

設定 enabled、timeout、環境變數或 headers

4. 驗證

用 opencode mcp list 或 prompt 檢查工具可用

5. 控權限

限制讀寫、網路、token 和高風險工具

OpenCode agent

- reads mcp config
- starts or connects MCP server
- receives tool list
- chooses tool call
- MCP returns observation
- agent uses result in answer or code change

MCP 不是模型本身的能力，而是把外部工具用標準協議交給 agent 使用。

設定 Local MCP Server

在 `opencode.jsonc` 中加入 `mcp` :

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "mcp_everything": {
      "type": "local",
      "command": ["npx", "-y", "@modelcontextprotocol/server-everything"],
      "enabled": true,
      "timeout": 5000
    }
  }
}
```

欄位：

- `type`: local
- `command`: 啟動 MCP server 的命令
- `environment`: 傳入環境變數
- `enabled`: 是否啟用
- `timeout`: 等待工具列表的時間

檢查 Local MCP 是否成功

啟動 OpenCode 後，先檢查 MCP server 是否載入。

```
opencode mcp list
```

或在 OpenCode 對話中要求：

```
請列出目前可用的 MCP server 和 tools。不要呼叫任何會修改資料的工具。
```

成功時應看到：

1. MCP server 名稱
2. 可用 tools 清單
3. 每個 tool 的用途或參數
4. 沒有 timeout 或 command not found 錯誤

Local MCP 失敗排查

若失敗，先檢查：

- `command` 是否可在 terminal 單獨執行
- Node / Python / Docker 是否已安裝
- `timeout` 是否太短
- 是否需要 proxy、token 或環境變數
- 是否把 config 放在正確位置
- 是否重啟 OpenCode

不要把 token 直接貼入 prompt；需要 token 時，優先用環境變數或安全的 secrets 管理方式。

設定 Remote MCP Server

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "internal-docs": {
      "type": "remote",
      "url": "https://mcp.example.edu/mcp",
      "enabled": true,
      "headers": {
        "Authorization": "Bearer YOUR_TOKEN"
      },
      "timeout": 5000
    }
  }
}
```

管理 MCP：

```
opencode mcp list
opencode mcp auth my-oauth-server
opencode mcp logout my-oauth-server
```

遠端 MCP 可能涉及外部網路、認證與資料傳輸；課堂示範應先用本地或測試 server。

OpenCode MCP 使用範例

Prompt 範例：

請使用 `mcp_everything` 工具列出目前可用工具。不要修改任何檔案。

請使用 `internal-docs` MCP 搜尋「安裝流程」，只根據 MCP 回傳結果回答，並列出來源。

請先說明你準備呼叫哪個 MCP tool、目的和輸入參數。等我確認後再執行。

良好習慣：

1. 先要求 agent 說明 tool call 計劃
2. 高風險工具設為 `ask` 或 `deny`
3. 不把 token 寫進 prompt
4. 不讓 MCP 讀取真實敏感資料
5. 對 MCP 回傳結果做來源與限制標示

MCP Prompt Pattern：先計劃，再呼叫

教學時建議要求 agent 分三段工作：

你可以使用 MCP tools，但必須遵守以下流程：

1. Tool Plan

- 說明你要呼叫哪個 MCP tool
- 說明目的
- 列出輸入參數
- 說明可能讀取或修改什麼資料

2. Wait for Approval

- 在我確認前，不要呼叫工具

3. Observation Summary

- 工具回傳後，只根據 observation 回答
- 標示來源、限制和仍需人工核對之處

適合高風險工具：

- 讀取資料庫
- 寫入檔案
- 呼叫內部 API
- 使用 token 的遠端服務
- 執行 shell command

OpenCode + MCP 權限範例

MCP tools 也應納入權限設計。可以先採保守策略：

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "*": "ask",
    "edit": "ask",
    "bash": {
      "*": "ask",
      "git status *": "allow",
      "rm *": "deny"
    }
  },
  "mcp": {
    "mcp_everything": {
      "type": "local",
      "command": ["npx", "-y", "@modelcontextprotocol/server-everything"],
      "enabled": false
    }
  }
}
```

OpenCode + MCP 教學建議

教學建議：

1. 預設 `enabled: false`
2. 需要示範時再啟用
3. 每次示範前說明 MCP 能力與風險
4. 結束後關閉或移除測試 MCP
5. 不把 token、密碼或真實敏感資料放入 prompt
6. 對遠端 MCP 先確認網路、權限與紀錄政策

七、選模型與硬體

小模型先跑通流程，再換大模型追求品質

模型選擇策略

用途	建議方向
課堂入門	1B-4B 小模型，確保所有電腦能跑
中文摘要	選中文能力較好的 instruct model
程式輔助	選 coding model 或 Qwen / DeepSeek coder 類模型
長文件	需要較大 context，並測試是否穩定
JSON 輸出	測試格式遵守能力，不要只看聊天品質
Tool calling	需要測試工具參數是否穩定生成

模型越大通常越慢、越吃記憶體；課堂 demo 應以「可穩定跑完」優先。

粗略硬體參考

模型大小	建議場景	粗略需求
1B-3B	入門、格式練習、簡短摘要	CPU 或普通筆電可試
4B-8B	一般聊天、中文摘要、簡單 coding	16GB RAM 較穩
14B+	較好推理與長文能力	建議 GPU / 大 RAM
30B+	高品質本地推理	高 VRAM 或 server 級硬體

注意：

1. 量化版本會影響速度與品質
2. context 越長越耗記憶體
3. 多人同時使用需要 server 設計
4. NPU / iGPU / GPU 支援視 runtime 和平台而定

課堂建議模型組合

低門檻：

```
ollama pull llama3.2:3b
```

中文與一般任務：

```
ollama pull qwen2.5:7b
```

較輕量多模態或一般用途：

```
ollama pull gemma3:4b
```

Coding demo：

```
ollama pull qwen2.5-coder:7b
```

實際模型名、tag 與可用性請以上課當日 Ollama model library 為準。

八、安全、治理與限制

本地模型降低部分風險，但不消除風險

本地 LLM 的安全邊界

風險	說明	控制方式
資料殘留	prompt、聊天記錄、模型 cache、應用 log 可能留在本機	使用測試資料，定期清理，了解 app 儲存位置
連接埠暴露	11434 若暴露到網路，其他人可能呼叫模型	只綁定 localhost，防火牆限制
模型幻覺	本地模型仍會編造內容	加來源、格式、覆核、停止條件
能力不足	小模型可能誤解長 prompt 或工具 schema	先做測試集，不適合就換模型
權限過大	coding agent 可讀寫檔案或執行命令	使用測試 repo，逐步批准變更
授權問題	模型 license 與資料使用限制不同	教學與部署前檢查 license

不要把本地 LLM 當成安全保證

錯誤想法：

因為模型在本機，所以任何資料都可以貼。

較好的做法：

1. 仍然使用去識別化資料
2. 不輸入真實個資或未公開案件資料
3. 不讓模型保留不必要的聊天歷史
4. 不開放 Ollama API 到公網
5. 不讓 coding agent 自動執行高風險命令
6. 對所有輸出做人工覆核

評估本地模型是否可用

每換一個模型，都用同一組測試題：

1. 繁體中文摘要是否自然？
2. 是否遵守表格或 JSON 格式？
3. 是否會承認資料不足？
4. 是否會編造來源？
5. 是否能處理本課警務文書 prompt？
6. 是否能穩定輸出 tool call arguments？
7. 回答速度是否可接受？

交付建議：

```
model:  
hardware:  
task:  
latency:  
format_following:  
hallucination_notes:  
recommended_use:  
not_recommended_use:
```

九、課堂實作

讓學生從命令列、API、應用整合三個層次都跑一次

Lab 1：CLI 測試

```
ollama pull llama3.2:3b  
ollama run llama3.2:3b
```

Prompt：

你是課程助教。請用繁體中文列出 local LLM 的 5 個優點和 5 個限制。
輸出格式必須是 Markdown 表格。

提交：

1. 模型名稱
2. 回答速度感受
3. 是否遵守表格格式
4. 一個你發現的錯誤或不足

Lab 2：API 測試

```
curl.exe http://localhost:11434/api/generate `
-H "Content-Type: application/json" `
-d "{\"model\":\"llama3.2:3b\",\"prompt\":\"用 JSON 列出三個 local LLM 風險。\",\"format\":\"json\",\"stream\":false}"
```

提交：

1. API 是否成功
2. 是否回傳 JSON
3. JSON 是否需要再清理
4. 如果失敗，錯誤訊息是什麼

Lab 3：接一個本地應用

三選一：

選項	任務
Open WebUI	開啟本地聊天介面，連接 Ollama
OpenCode	在測試 repo 中讓 agent 只讀 README，不修改檔案
Continue	在 IDE 中讓本地模型解釋一個小函式

提交：

- 使用應用：
- 連接模型：
- 成功做到：
- 遇到問題：
- 安全注意：

Lab 4：警務文書 prompt 測試

把本課前面使用過的去識別化資料貼給本地模型：

報案人稱在網上購買門票，轉帳後對方失聯。報案人有聊天截圖和轉帳紀錄，但不知道對方真實身份。

要求：

請整理成：

1. 已知事實
 2. 未核實資訊
 3. 證據來源
 4. 待補問問題
 5. 不可下結論之處
- 不得判斷犯罪是否成立。

比較本地模型與雲端模型輸出的差異。

常見問題排查

問題	檢查
<code>ollama</code> command not found	是否已安裝並重開 terminal
API 無法連線	Ollama 是否正在執行；檢查 <code>localhost:11434</code>
模型很慢	換小模型；減少 context；檢查 GPU
記憶體不足	使用 1B-4B 模型或較高量化版本
應用找不到模型	先 <code>ollama pull</code> ，再檢查 model name
OpenCode tool call 不穩	換模型、加大 context、簡化任務
中文輸出差	換中文能力較好的模型

課堂總結

你應該會做

- 安裝 Ollama
- 下載與執行本地模型
- 用 API 呼叫模型
- 把模型接到本地應用
- 測試模型能力與限制

你應該保持警覺

- 本地不等於無風險
- 小模型能力有限
- coding agent 有檔案權限風險
- 正式資料仍需去識別化
- 輸出仍需人工覆核

Local LLM 的價值，是讓我們在可控環境中學習、測試和原型設計；不是讓模型繞過資料治理或人工責任。

參考資料

類別	來源
Ollama	https://ollama.com/
Ollama API	https://github.com/ollama/ollama/blob/main/docs/api.md
Model library	https://ollama.com/library
OpenCode providers	https://opencode.ai/docs/providers/
OpenCode agents	https://opencode.ai/docs/agents/
OpenCode permissions	https://opencode.ai/docs/permissions/
OpenCode MCP	https://opencode.ai/docs/mcp/
OpenCode Skills	https://opencode.ai/docs/skills/
DigitalOcean Agent Skills	https://www.digitalocean.com/community/tutorials/how-to-implement-agent-skills
Anthropic Claude Skills Guide	https://resources.anthropic.com/hubfs/The-Complete-Guide-to-Building-Skill-for-Claude.pdf
Hugging Face MCP Course	https://huggingface.co/learn/mcp-course/unit1/introduction
HF MCP Unit 1 Key Concepts	https://huggingface.co/learn/mcp-course/unit1/key-concepts
HF MCP Unit 1 Architecture	https://huggingface.co/learn/mcp-course/unit1/architectural-components
HF MCP Unit 1 Protocol	https://huggingface.co/learn/mcp-course/unit1/communication-protocol
HF MCP Unit 1	https://huggingface.co/learn/mcp-course/unit1/evaluation